Contents

1 Stacks *3*

1.1 Array-Based Stack *4*

1.2 Linked-List Stack *5*

1.3 Array vs. Linked: Which One? *6*

2 Queues *7*

2.1 Array-Based Queue *7*

2.2 Linked-List Queue *8*

3 Priority Queues *9*

3.1 A Naive Implementation of the Priority Queue *9*

3.2 Using the Priority Queue *10*

3.3 Binary Search *12*

3.4 Algorithm *13*

4 Hash Tables *15*

4.1 Structure of a Hash Table *15*

4.2 Inserting and Retrieving Data *15*

4.3 Handling Collisions *15*

4.4 Time Complexity *16*

5 Trees *17*

5.1 Binary Trees and Binary Search Trees *18*

5.2 File Structures and File System Hierarchies *20*

5.3 Red-Black Trees and AVL Trees *22*

5.4 B-Trees, Tries, and Suffix Trees 22

6 Tree Traversal 23

6.1 BFS 23

6.1.1 Pseudocode 25

6.2 DFS 26

6.2.1 Pre-order 26

6.2.2 Post-order 27

6.2.3 In-order 27

6.2.4 Pseudocode 28

6.3 Recursive vs Iterative Traversal 28

6.3.1 Pre-order and Post-order Numbering 29

7 Finding Shortest Paths 31

7.1 Introduction — BFS revisited 31

7.2 Dijkstra's Algorithm 32

7.2.1 Algorithm Description 32

7.2.2 Implementation 35

7.3 Bellman-Ford Algorithm 38

8 Other Graph Algorithms and Proofs 41

8.1 Adjacency Matrices 41

8.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees 44

8.3 A* (A-star) Search Algorithm 44

8.3.1 The Path Cost $g(n)$ 44

8.3.2 The Heuristic Function $h(n)$ 45

8.3.3 The Total Estimated Distance $f(n)$ and Priority Queues 46

8.4 Injection, Surjection, Bijection, and Isomorphism 46

8.5 Travelling Salesperson Problem and Nearest Neighbour 50

8.6 Topological Sort and DAGs 50

8.7 Proofs 53

8.7.1 Example 1: Bipartite 53

8.7.2 Example 2 and 3. Cycle Detection and Leaves 54

A Answers to Exercises 57

Index 63

Chapter 1

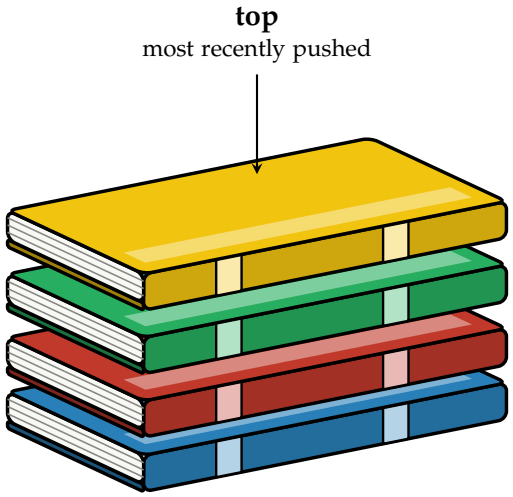
Stacks

A *stack* is a linear data structure that only allows access at one end, called the *top*.

Data comes off in the reverse order it went on: the last thing pushed onto the stack is the first thing popped off. This ordering is called *LIFO*, for "last in, first out."

You have already seen a stack in a previous workbook, even if it wasn't called one: the call stack from the Introduction to C chapter behaves exactly this way, with each function call pushing a new frame on top and returning popping it back off.

We can think of this like a stack of books: you can only access the book on top, and you can only see the cover of the top book. If you want to get to a book underneath, you have to remove the books above it first.



Every stack, no matter how it is built underneath, supports the same handful of operations:

- `push(value)` put a new value on top of the stack.
- `pop()` remove and return the value on top of the stack.
- `peek()` look at the value on top of the stack, without removing it.
- `isEmpty()` true if the stack has nothing in it.

Notice that we can only use the value on top of the stack, no matter how large it is.

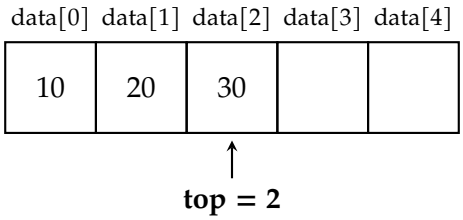
We are going to build a stack two different ways: once backed by a fixed-size array, and once backed by the linked-list nodes from the previous chapter. Both give you `push`/`pop`/`peek`/`isEmpty`, but they make very different tradeoffs underneath.

1.1 Array-Based Stack

The simplest way to build a stack is on top of a plain array. We keep a fixed-size buffer and a single integer, `top`, that tracks the index of the top element. An empty stack is represented by `top == -1`.

```
const int CAPACITY = 5;

struct ArrayStack {
    int data[CAPACITY];
    int top;   // index of the top element; -1 means empty
};
```



`push` and `pop` only ever touch the `top` index and whatever cell it points at, so both run in constant time, $O(1)$. The catch is `CAPACITY`: once the array is full, there is nowhere left to push, so a correct implementation has to check for and reject that case rather than writing past the end of the array.

```
bool isEmpty(const ArrayStack& s) {
    return s.top == -1;
}

bool isFull(const ArrayStack& s) {
    return s.top == CAPACITY - 1;
}

void push(ArrayStack& s, int value) {
    // prevent pushing onto a full stack
    if (isFull(s)) {
        std::cout << "push(" << value << ") failed: stack overflow" << std::endl;
        return;
    }
    s.top++;
```

```
    s.data[s.top] = value;
}

int pop(ArrayStack& s) {
    // prevent popping from an empty stack
    if (isEmpty(s)) {
        std::cout << "pop() failed: stack underflow" << std::endl;
        return -1;
    }
    int value = s.data[s.top];
    s.top--;
    return value;
}
```

Notice that `pop` does not actually erase `s.data[s.top]`, it just moves `top` down by one. The old value is still sitting in the array, but nothing can reach it anymore, since every operation only ever looks at indices `0` through `top`.

The full, runnable version of this file, including a `main()` that demonstrates both overflow and underflow, is in your digital resources. In this version, we added memory address output to demonstrate that shows how the array is a contiguous block.

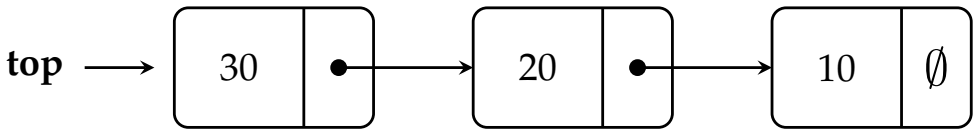
1.2 Linked-List Stack

The array-based stack has one real weakness: `CAPACITY` is fixed at compile time. If you don't know in advance how much data you'll be pushing, you either waste memory reserving more than you need, or you risk overflow. A stack built on the `Node` type from the Linked Lists chapter has no limit, since it grows and shrinks on the heap as needed. The downside is that each element takes more memory, since it has to store a pointer to the next node.

```
struct Node {
    int data;
    Node* next;
};

struct LinkedStack {
    Node* top;   // nullptr means empty
};
```

Instead of an array index, `top` is now a pointer to the first node. Pushing means creating a new node whose `next` points at the old top, then moving `top` to the new node; popping is the reverse.




```cpp
bool isEmpty(const LinkedStack& s) {
    return s.top == nullptr;
}

void push(LinkedStack& s, int value) {
    Node* n = new Node();
    n->data = value;
    n->next = s.top;    // the new node points at the old top
    s.top = n;          // the new node becomes the top
}

int pop(LinkedStack& s) {
    if (isEmpty(s)) {
        std::cout << "pop() failed: stack underflow" << std::endl;
        return -1;
    }
    Node* oldTop = s.top;
    int value = oldTop->data;
    s.top = oldTop->next;
    delete oldTop;
    return value;
}
```

Both `push` and `pop` still run in constant time (we will cover this in a future chapter), same as the array version, since neither one has to walk the list. But unlike the array version, `pop` here calls `delete` on the old top node; if it didn't, the node would sit on the heap forever.

The full version of this file is in your digital resource!

1.3 Array vs. Linked: Which One?

	Array-Based	**Linked-Based**
Max size	Fixed at compile time	Limited only by available memory
Memory per element	Just the value	Value + one pointer

If you know a safe upper bound on how much data you'll ever hold, the array version is the most efficient* choice. If you don't, the linked version is safer, since it will never overflow.

Chapter 2

Queues

Imagine you are at a concert. You are arriving at around 1pm, but the line has been growing since about 3am that same day (wow!). Naturally, by way of timing, the people who arrived earliest are going to be able to enter the venue first, while the people you arrive closest to the show time will enter the venue last.

Assuming supplements like VIP packages or fast-track lines are not of concern here, we can summarize this type of line in a similar phrase: **First In, First Out**. The first people to arrive in the line are the first people out of the line, and, subsequently, into the venue. We call this way of organizing data *FIFO*.

Similar to stacks, we only need a handful of operations to build a queue.

[placeholder: people in line image (maybe)]

Every stack, no matter how it is built underneath, supports the same handful of operations:

- `enqueue(value)` put a new value at the front of the queue. You may see this as `enq`
- `dequeue()` remove and return the value at the front of the queue. You may see this as `deq`
- `peek()` look at the value on front of the queue
- `isEmpty()` true if the queue has nothing in it.

Similarly, again, to stacks, we can implement this two ways: a fixed-array based method and a linked-list method. Both have benefits and tradeoffs.

2.1 Array-Based Queue

We can build a queue with an array, there is the same major constraint as with stacks: we need to know the size of the queue in advance.

```
1  const int CAPACITY = 5;
2
3  struct ArrayQueue {
4      int data[CAPACITY];
5      int front;   // index of the front element; -1 means empty
```

```
    int count = 0; // for ensuring we don't exceed CAPACITY

};

bool enqueue(Queue& q, int value) {
    if (isFull(q)) return false;
    int back = (q.front + q.count) % MAX;  // wraps around
    q.data[back] = value;
    q.count++;
    return true;
}

bool dequeue(Queue& q, int& out) {
    if (isEmpty(q)) return false;
    out = q.data[q.front];
    q.front = (q.front + 1) % MAX;         // wraps around
    q.count--;
    return true;
}

bool peek(const Queue& q, int& out) {
    if (isEmpty(q)) return false;
    out = q.data[q.front];
    return true;
}
```

TODO: Explain why we need to wrap around the array (circular array) and how it is done. Simplify above code.

2.2 Linked-List Queue

Chapter 3

Priority Queues

As mentioned in the last chapter, you are going to make a priority queue for use with Dijkstra's Algorithm. Using it will look like this algorithm called a *priority queue*:

```
import kpqueue

myqueue = pqueue.PriorityQueue()
myqueue.add(long_beach, 0) # Inserts first city and its cost
myqueue.add(san_diego, 14) # Puts San Diego after Long Beach
myqueue.add(los_angeles,12) # Inserts LA between Long Beach and San Diego
current_city = myqueue.pop() # Returns first city (Long Beach) and removes it
```

Now, if an item gets a new priority, we need to remove it and reinsert it in the new spot.

```
myqueue.add(city_a, 16) # Puts it last in the queue
myqueue.update(city_a, 16, 13) # Moves it to between LA and San Diego
```

3.1 A Naive Implementation of the Priority Queue

Create a file called `kpqueue.py`. Let's do a simple implementation that stores the priority and the data as tuple. And we will keep it sorted by the priority. If two tuples have the same priority, we will sort by the data.

Type this in to `kpqueue.py`:

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)
```

```
14
15      def update(self, value, old_priority, new_priority):
16          old_pair = (old_priority, value)
17          self.list.remove(old_pair)
18          self.add(value, new_priority)
19
20      def add(self, value, priority):
21          pair = (priority, value)
22          # Add it at the end
23          self.list.append(pair)
24          # Resort the list
25          self.list.sort()
```

This will work fine, but it could be much more efficient:

- Every time we add a single element, we resort the whole list.
- The function `remove` is searching the list sequentially for the item to delete.

In a minute, we will revisit these inefficiencies and make them better.

3.2 Using the Priority Queue

We are going to change `graph.py` to use the priority queue. While we are doing so, why don't we also shrink the memory footprint of our program a bit.

Notice that as the algorithm is running, each node is in one of three states:

- Unseen: In the earlier implementation, these were the nodes with `math.inf` as their cost.
- Seen, but not finalized: These are "unvisited", but don't have `math.inf` as their cost.
- Finalized: These are the "visited" nodes — we know that their cost won't decrease any more.

We can shrink the memory foot print by not putting the unseen into the dist dictionary at all. And instead of a separate set for "unvisited", what if we moved finalized nodes and their distances into a separate dictionary?

Rewrite the `cost_from_node` function in `graph.py`:

```
1               # Visited nodes are already minimized, skip them
2               if v in finalized_dist:
3                   continue
```

```
                # What is the cost to this neighbor?
                alt = current_node_cost + edge.cost

                # Is this the first time I am seeing the node?
                if v not in seen_dist:

                    # Insert into the seen_dict, prev, and priority queue
                    seen_dist[v] = alt
                    prev[v] = current_node
                    pqueue.add(v, alt)

                else: # v has been seen. Is this a cheaper route?
                    old_dist = seen_dist[v]
                    if alt < old_dist:
                        # Update the seen_dict, prev, and priority queue
                        seen_dist[v] = alt
                        prev[v] = current_node
                        pqueue.update(v, old_dist, alt)

        return (finalized_dist, prev)
```

This should be have exactly the same result, except for the unreachable nodes. If you have a graph that is not connected, there will be nodes that cannot be reached from the origin. In the old version, these had a cost of `math.inf`. Now, they will just not be in the dictionary at all. So, change `cities.py` to deal with this:

```
if nyc in cost_from_long_beach:
    nyc_cost = cost_from_long_beach[nyc]
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"\n*** Cheapest path from Long Beach to NYC: {path_to_nyc} ***")
else:
    print("You can't get to NYC from Long Beach")
```

If you run `cities.py` now, it should behave exactly like the old version.

However, there is a bug. It will rear its head if two cities with the same cost are in the priority queue together. Change `cities.py` so that Denver and Phoenix have the same cost:

```
graph.WeightedEdge(12, long_beach, los_angeles)
graph.WeightedEdge(19, los_angeles, denver)
graph.WeightedEdge(19, los_angeles, phoenix)
```

Now try running it. You should get an error:

```
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

What happened? The `loc_for_pair` method is comparing tuples made up of a `float` and a `Node`. The `float` comes first in the tuple, so that is compared first. However, if the two tuples have the same priority, it then compares nodes.

The error statement says, "Nodes don't have a less-than method; I don't know how to compare them."

Each `Node` lives at an address in memory. You can get that address as a number using the `id` function. The ID is unique and constant over the life of the object. It is a rather arbitrary ordering, but it will work for this problem. Add a method to your `Node` class:

```
    # Nodes will be ordered by their location in memory
    def __lt__(self, other):
        return id(self) < id(other)
```

Fixed.

Now, let's make the priority queue more efficient.

3.3 Binary Search

The phone company in every town used to print a thing called a phone book. The names and phone numbers were arranged alphabetically. As you might imagine, these books often had more than a thousand pages.

If you were looking for "John Jeffers", you would not start at the first page and read sequentially until you reached his name. You would open the book in the middle, and see a name like "Mac Miller", then think "Jeffers comes before Miller". You would then split the pages in your left hand in half and see a name like "Hester Hamburg" and think "Jeffers comes after Hamburg". You would then split the pages in your right hand, and continue on like this until you found the page with "John Jeffers" on it. That is binary search.

Binary Search is a search algorithm that finds the position of a target value within a sorted array. The binary search algorithm works by repeatedly dividing the search interval in half. If the target value is equal to the middle element of the array, the position is returned. If the target value is less than or greater than the middle element, the search continues in the lower or upper half of the array, respectively.

3.4 Algorithm

The binary search algorithm can be described as follows:

1. If the array is empty, the search is unsuccessful, so return "Not Found".
2. Otherwise, compare the target value to the middle element of the array.
3. If the target value matches the middle element, return the middle index.
4. If the target value is less than the middle element, repeat the search with the lower half of the array.
5. If the target value is greater than the middle element, repeat the search with the upper half of the array.
6. Repeat steps 2-5 until the target value is found or the array is exhausted.

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)

    def add(self, value, priority):
        pair = (priority, value)
        i = self.loc_for_pair(pair)
        self.list.insert(i, pair)

    def update(self, value, old_priority, new_priority):
        old_pair = (old_priority, value)
        i = self.loc_for_pair(old_pair)
        del self.list[i]
        self.add(value, new_priority)

    def loc_for_pair(self, pair):
        # The range where it could be is [lower, upper)
        # Start with the whole list
        lower = 0
        upper = len(self.list)

        while upper > lower:
            next_split = (upper + lower) // 2
```

```
            v = self.list[next_split]
            if pair < v:  # pair is to the left
                upper = next_split
            elif pair > v:  # pair is to the right
                lower = next_split + 1
            else: # Found pair!
                return next_split
        return lower
```

If you try running it now, it should work perfectly.

You now have a graph class that will find the cheapest path quickly, even if it has thousands of nodes with thousands of edges.

Chapter 4

Hash Tables

A hash table, also known as a hash map, is a data structure that implements an associative array abstract data type, a structure that can map keys to values. It uses a hash function to compute an index into an array of buckets or slots, from which the desired value can be found.

4.1 Structure of a Hash Table

A hash table is composed of an array (the 'table') and a hash function. The array has a predetermined size, and each location (or 'bucket') in the array can hold an item (or several items if collisions occur, as will be discussed later). The hash function is a function that takes a key as input and returns an integer, which is then used as an index into the array.

4.2 Inserting and Retrieving Data

When inserting a key-value pair into the hash table, the hash function is applied to the key to compute the index for the array. The corresponding value is then stored at that index.

When retrieving the value associated with a key, the hash function is applied to the key to compute the array index, and the value is retrieved from that index.

4.3 Handling Collisions

A collision occurs when two different keys hash to the same index. There are several methods for handling collisions:

- **Chaining (or Separate Chaining)**: In this method, each array element contains a linked list of all the key-value pairs that hash to the same index. When a collision occurs, a new key-value pair is added to the end of the list.
- **Open Addressing (or Linear Probing)**: In this method, if a collision occurs, we move to the next available slot in the array and store the key-value pair there. When looking up a key, we keep checking slots until we find the key or reach an empty slot.

4.4 Time Complexity

In an ideal scenario where hash collisions do not occur, hash tables achieve constant time complexity $O(1)$ for search, insert, and delete operations. However, due to hash collisions, the worst-case time complexity can become linear $O(n)$, where n is the number of keys inserted into the table.

Using good hash functions and collision resolution strategies can minimize this issue and allow us to take advantage of the hash table's efficient average-case performance.

Chapter 5

Trees

Trees are one of the most versatile and widely used data structures in computer science. A *tree* is a hierarchical data structure consisting of nodes, where each node has a value and a set of references (*edges*) to its child nodes. The node at the top of the hierarchy is called the *root*, and nodes with the same parent are called *siblings*, as seen in Figure 5.1.

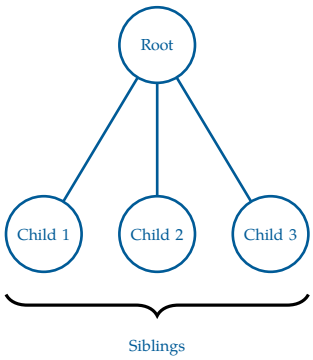


Figure 5.1: A tree with a root node and 3 children.

Trees are a subset of Graphs, a data structure comprised of nodes and edges. Graphs can be converted into trees by adding the following two constraints:

- The graph is connected, such that there exists a path between every pair of nodes
- The graph is acyclic; no cycles exist in the graph

With this comes the caveat that the number of edges in a tree is edges $= n - 1$, where n is the number of nodes in the tree. This is because a tree with n nodes must have exactly $n-1$ edges to maintain its connected and acyclic properties. If there were more than $n - 1$ edges, it would create a cycle, violating the acyclic property. If there were fewer than $n - 1$ edges, it would not be fully connected, violating the connected property.

Less formally, a *forest* is a collection of multiple trees (may be disconnected), and a *tree* is a forest with only one tree. Lots of agricultural terms in this chapter!

Trees have a unique structural property: between any two nodes, there exists exactly one simple path. This follows directly from the fact that trees are both connected and acyclic. As a result, there is only one way to move from a start node to an end node.

Although trees are often drawn with a root at the top, this is a matter of convention. Any node in a tree can be chosen as the root, and doing so induces a hierarchical structure in which edges are interpreted as parent-child relationships.

Once a root is selected, every node (except the root) has exactly one parent, and zero or more children. Because of this relationship, any randomly selected node can form a tree (we call this a *subtree*, consisting of the node and all its descendants), making the tree structure *recursive*. Moving from one node to the next is the recursive step that minimizes

the size.

Nodes with no children are called, as you'd expect, *leaves*, as they are the outermost node of a tree. The *depth* of a chosen node is the number of edges from the *root* to that node, and the *height* of a tree is the maximum depth among all nodes. Equivalent to the height of a tree is the maximum number of edges from the *root* to any leaf. You may see the *level* of a node referred to as the number of edges from the *root* to that node, similar to depth. By convention, the level and depth of the root is 0. See Figure 5.2.

Some more relevant terminology you may encounter in the context of trees:

- *Path*: A sequence of nodes where each adjacent pair is connected by an edge. This can be used to show the idea of how to get between two nodes in a tree, for example, between two cities in a flight path network.
- *Ancestor*: A node that lies on the path from the root to a given node. This is done by repeatedly traversing from a child node to a parent node until the selected node is reached.
- *Descendant*: A node that lies on the path from a given node to a leaf. This is done by repeatedly traversing from a parent node to a child node until a leaf is reached.

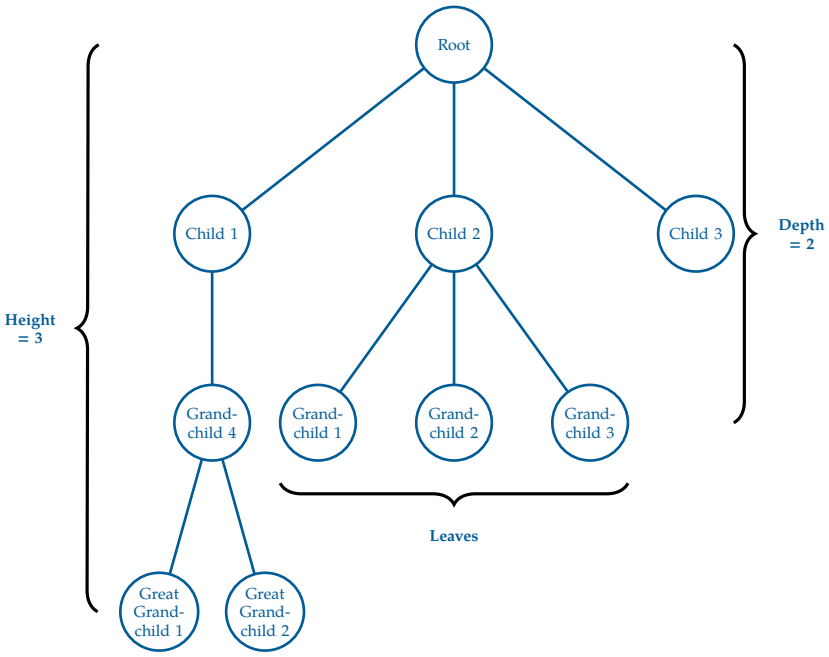


Figure 5.2: A tree with a root node and 3 children, a few grandchildren, and 2 great grandchildren.

FIXME tree proof

5.1 Binary Trees and Binary Search Trees

The most common type of tree is the *binary tree*, where each node has *at most* two children, referred to as the *left child* and the *right child*. Binary trees are widely used in various applications, including expression parsing, binary search trees, and heaps.

Let's put this practice. Recall our binary search implementation from the recursion chapter. Given a chosen word and a sorted list of words, such as a dictionary, we can find the page of the chosen word by repeatedly dividing the available pages in half and comparing

the elements on the page to the chosen word. This is much easier than, for example, going through every single page and checking if the word is on that page.

Say we wanted to represent our dictionary as a tree that we could recursively binary search through. For simplicity, we will limit our words to `mango`, `apple`, `pear`, `banana`, `anagram`, `peony` and `peach`. These 7 words form a binary tree of height 2, as seen in Figure 5.3. We select the word `mango` as the root as it is the middle word after sorting alphabetically.

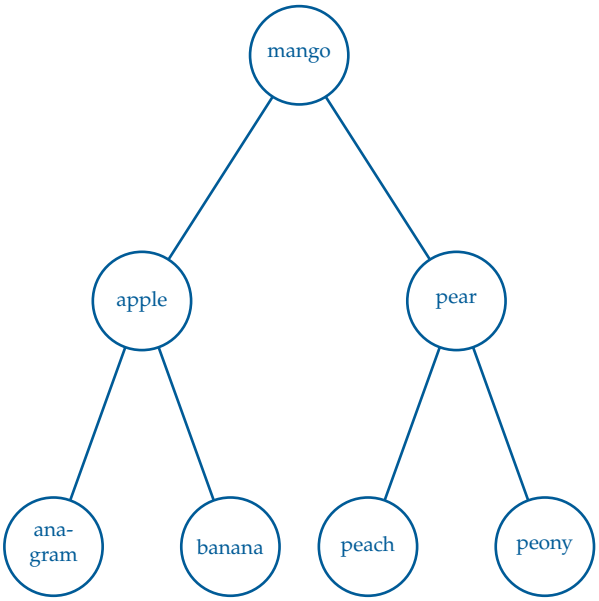


Figure 5.3: A binary search tree with 7 words.

Similar to binary search, we can find any word in the tree by starting at the root and comparing the target word to the current node's word. If the target word is less than the current node's word, we move to the left child; if it is greater, we move to the right child. We repeat this process until we find the target word or reach a leaf node, which tells us that the target word is not in the tree. This structure allows us to cut the available words in half with each step in the tree we go.

What if we want to insert into a binary search tree? We must complete four actions:

- Find the correct location for the new node.
- Create and insert the node at that location,
- Connect it to its respective parent(s).
- Ensure the BST properties are maintained.

Let's say, on our tree above, we want to add `orange` to the tree. Because of alphabetical sorting,

FIXME expand on addition, removal, and search of binary trees FIXME add exercise of a similar dictionary tree with more words, have student add and remove words, and search for words

5.2 File Structures and File System Hierarchies

Computers, since their first creation in the 1980s, have needed a way to store and organize data. Prior to computers, many systems of organization were used to store physical data, such as filing cabinets, libraries, and warehouses.

These systems often had a hierarchical structure, with a top-level category that branched out into subcategories, with these subcategories further branching out into more specific categories, and so on.

For example, a library might have two top-level categories: `fiction` and `non-fiction`. Within the category `non-fiction`, you may find `psychology`, `biographies`, `autobiographies`, `self-help`, `journalism` books, or even `nature guides`. Under the category `fiction`, you may find `romance`, `classics`, `crime`, `science fiction`, `thriller`, or `fantasy` novels. Of course, these sub-genres can have even further categorizations too.

A hospital may have patient records categorized by first letter of last name, and then sorted alphabetically from there.

With the development of the computer, sorting these types of hierarchical data became easy with the concept of directories. A *directory*, also called a *folder*, is a categorical way of organizing data (for example, by storing files or other subdirectories). We will stick to the more technical term of directories, however, any time you create a folder on your computer, you are making a *directory*. The name comes from how, in the twentieth-century, many phone books were referred to as telephone directories.

This structure naturally fits into the tree data structure:

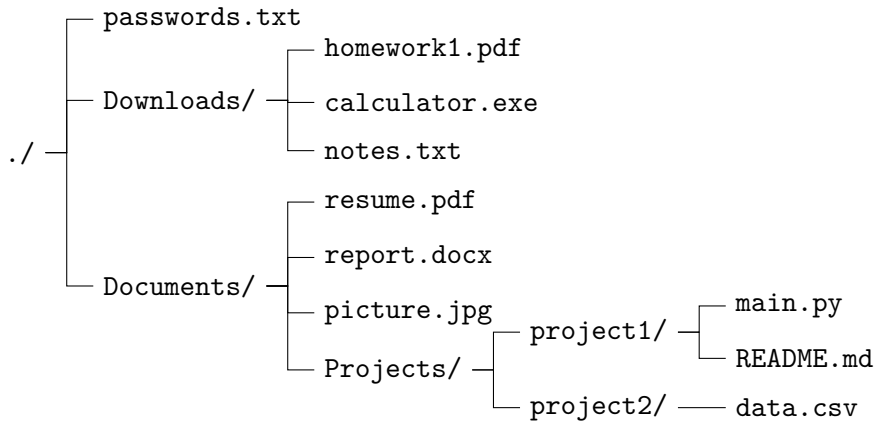
- The **root** of the tree is the top-level directory (`/` on Linux/Unix[1] systems or `C:\` on Windows).
- A **node** is naturally fits the idea of either *directories nodes* or *files nodes*.
- A **directory node** may have children, where its children are subdirectories or files.
- A **file node** is a **leaf**, meaning it has no children.

The **path** to a file describes the unique sequence of directories from the root to that file. Test again test again test again

Example

A small file system might look like:

[1] There is a very helpful video in Linux File Systems in your digital resources. Check it out!



In tree terminology:

- `./` is the root node.
- `Downloads`, `Documents`, and its subdirectories are internal nodes (directories).
- All files, such as those ending in `.txt`, `.png`, `.exe`,`.csv`, `.docx`, `.py` files are leaf nodes.

Side note: please do not store your passwords in anywhere on your computer's memory!

Exercise 1 File Paths

Working Space

In the above File Hierarchy Tree, what is the path to `data.csv`? What is the depth of the file tree at that point?

Answer on Page 57

5.3 Red-Black Trees and AVL Trees

[FIXME add content on red black trees and AVL trees]

5.4 B-Trees, Tries, and Suffix Trees

[FIXME add content on B-trees, tries, and suffix trees]

Chapter 6

Tree Traversal

Trees can be "traversed" in a few different ways. When we say traversing a tree, we mean that we are visiting and extracting data from each node in a tree. We classify these *traversals* based on the order in which the nodes are visited.

This chapter will discuss *breadth-first search*, also known as *BFS*, which traverses in a *level-order*. Each level is traversed in sequential order.

Alternatively, there is *depth-first search* (*DFS*), which travels as far down a branch as possible before backtracking and checking other branches. Within DFS there are 3 common traversal methodologies: *in-order*, *pre-order* and *post-order*.

While the demonstrations for these traversals will be for trees, they work the same for any given graph, under one condition: using DFS or BFS on a *graph* requires you do have a preselected node a your *start node*.

6.1 BFS

Let's look at running BFS on a tree. Starting out with this tree:

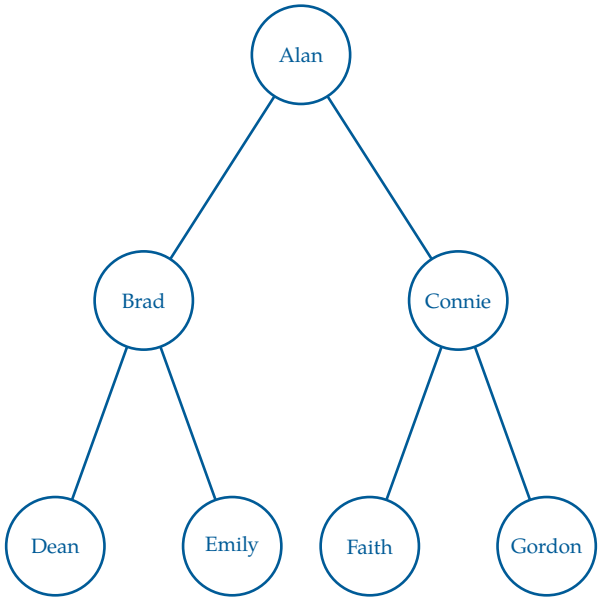


Figure 6.1: Our original tree containing a list of names to be traversed.

To perform BFS, we use a *queue*, which follows a first-in, first-out (*FIFO*) order. We begin by placing the root node into the queue. At each step, we remove the front node from the queue, visit it, and then add all of its children to the *back* of the queue.

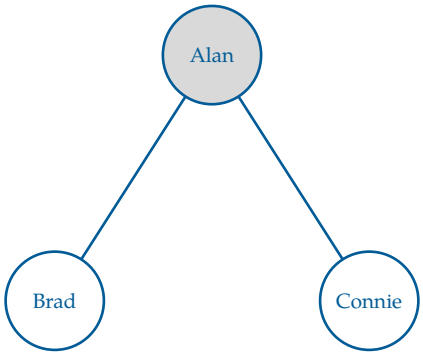


Figure 6.2: Step 1: Visit Alan, then add its children (Brad, Connie) to the queue.

Here, we visit the root node first, which is Alan. After visiting Alan, we add its children, Brad and Connie, to the queue. Since there are no other nodes to visit at this level, we move on to the next level of the tree.

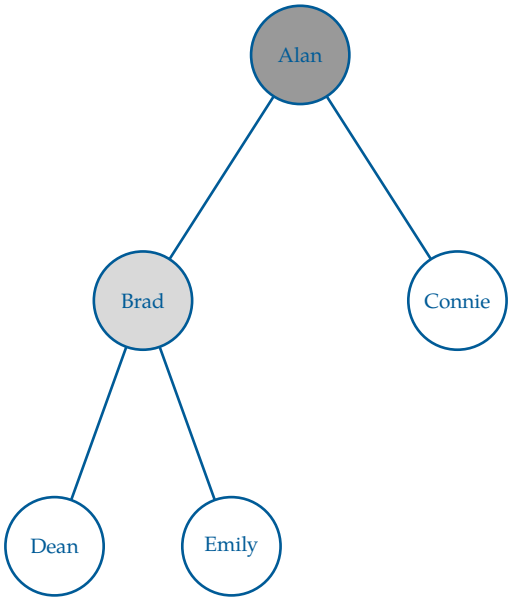


Figure 6.3: Step 2: Visit Brad, then add its children (Dean, Emily) to the queue.

We continue this process, visiting Connie next and adding its children, Faith and Gordon, to the queue. The queue now contains Connie, Dean, and Emily. Here is a step trace of the BFS traversal:

Step	Queue	Output (visited)
Start	[Alan]	[]
1	[Brad, Connie]	[Alan]
2	[Connie, Dean, Emily]	[Alan, Brad]
3	[Dean, Emily, Faith, Gordon]	[Alan, Brad, Connie]
4	[Emily, Faith, Gordon]	[Alan, Brad, Connie, Dean]

Continuing this process until the queue is empty gives the final traversal:

Alan, Brad, Connie, Dean, Emily, Faith, Gordon

This ordering reflects a level-by-level traversal of the tree, which is why BFS on a tree is also called level-order traversal.

To use BFS for *searching* rather than full traversal, we simply introduce a target value and stop the algorithm as soon as the target is found. Instead of visiting every node, we check each node when it is dequeued and terminate early if it matches the desired value.

For example, suppose we want to find "Faith" in the tree. BFS will examine nodes level by level: Alan, then Brad and Connie, then Dean, Emily, and Faith. As soon as Faith is encountered (we *dequeue* it from the queue), the search stops, without visiting any remaining nodes.

6.1.1 Pseudocode

Our steps can be reflected as pseudocode as follows:

Algorithm 1 Breadth-First Search (BFS) - Iterative

```
procedure BFS(root, target)
    Q = empty queue
    ENQUEUE(Q, root)
    while Q is not empty do
        node = DEQUEUE(Q)
        if node = target then
            return node
        end if
        for each child of node do
            ENQUEUE(Q, child)
        end for
    end while
    return null
end procedure
```

This procedure is often used in game applications to find paths, such as maze solvers, chess algorithms, and tic-tac-toe solutions. It is also used in social network analysis to find the shortest path between two users, and in web crawling to explore the structure of websites.

6.2 DFS

In contrast, we can run a *depth-first search* on the original tree (see Figure 6.1). Depth-first search, as the name implies, goes as deep down the tree until reaching a leaf, and then backtracks to the next possible, unsearched path. DFS (implicitly) uses a stack to explore the graph or tree.

The steps are fairly simple:

1. Push the root node into the stack.
2. Pop a node from the stack, and mark it as visited.
3. Push the node's children onto the stack.
4. Repeat steps 2 and 3 until the stack is empty.

There are 3 orders in which we can traverse/search the deepest tree. We will look at `pre-order`, `post-order`, and `in-order`.

However, these orders essentially perform the same basic three tasks,

V Visit the selected node.

L Run DFS on the selected node's left subtree.

R Run DFS on the selected node's right subtree.

The recursion terminates when we reach a leaf. At this point, the algorithm returns to the previous call, which is how *backtracking* occurs. This backtracking is done via the stack we had mentioned previously.

6.2.1 Pre-order

In `pre-order` traversal, we visit the node *before* exploring its subtrees. The order is:

$$(V, L, R)$$

We visit the node *before* its children because the node's value is needed *prior* to exploring its subtrees. This is useful when the node represents a decision that affects which branch of the tree you go down, such as in tree copying or directory traversal like we talked about in the previous chapter.

Using our original tree, the pre-order results are:

Alan, Brad, Dean, Emily, Connie, Faith, Gordon

6.2.2 Post-order

In `post-order` traversal, we visit the node *after* exploring both subtrees. The order is:

$$(L, R, V)$$

We visit the node *after* its children because the node's computation depends on results from both subtrees. This is useful when combining subtree results or deleting trees, where children must be handled before the parent.

The post-order traversal of our original tree is

Dean, Emily, Brad, Faith, Gordon, Connie, Alan

6.2.3 In-order

In `in-order` traversal, we visit the node *between* exploring its subtrees. The order is:

$$(L, V, R)$$

We visit the node *between* its subtrees because the correct interpretation of the node depends on partially processed children. In-order only makes sense for binary trees, because it returns the *sorted order* of the binary search tree. It depends on left and right subchildren. This would be like the alphabetical order of our dictionary example from the previous chapter.

The in-order traversal of our original tree becomes

Dean, Brad, Emily, Alan, Faith, Connie, Gordon

6.2.4 Pseudocode

The previous sections (pre-order, in-order, and post-order) describe *when* a node is visited during a depth-first traversal.

The following algorithm instead describes the general *mechanism* of depth-first search. This implementation corresponds to a pre-order traversal. Other traversal orders (in-order and post-order) require additional state and cannot be obtained by simple reordering alone.

Algorithm 2 Depth-First Search (DFS) - Iterative on a Tree

```
procedure DFS(root)
    S = empty stack
    Push(S, root)
    while S is not empty do
        node = Pop(S)
        visit(node)                                  ▷ V
        if node.left ≠ null then
            Push(S, node.left)                       ▷ L
        end if
        if node.right ≠ null then
            Push(S, node.right)                      ▷ R
        end if
    end while
end procedure
```

DFS is particularly useful for solving problems like connected-component detection in graphs and maze-solving.

6.3 Recursive vs Iterative Traversal

As we've established, trees are recursively defined structures, and as such, our algorithms for them can be defined as such too. This allows us to implement searches and traversals in two ways: *recursive* and *iterative*. Let's look at DFS recursively.

Importantly, they do not change the traversal order (pre-, in-, post-order); they only change how the traversal is implemented.

A *recursive traversal* relies on the program's *call stack* (recall from the Advanced Python Syntax chapter). Each time the algorithm explores a subtree, it makes a function call, and the system automatically remembers where to return when that call finishes. This matches the recursive definition of a tree, where each subtree is itself a tree. This allows us to get rid of the stack in DFS and solely rely on a computer's call stack.

Exercise 2 Recursive DFS

Take a look at the Iterative DFS code above.

How can we rewrite the pseudocode to be recursively define, taking advantage of a inherent recursive structure? Try pre-order recursively first. Assume the tree is a binary tree.

Working Space

Answer on Page 57

Note that we can create recursive DFS just by reordering the three operations (`V`, `L`, `R`).

What about BFS? BFS is typically implemented iteratively because it relies on a queue. While recursive versions exist, they still depend on a queue and offer little conceptual advantage, so we omit them here.

6.3.1 Pre-order and Post-order Numbering

There also exists pre-order and post-order numbering, regardless of which version of DFS is run. Pre-order and post-order numbering are techniques used during a depth-first search (DFS) traversal to record when each node is encountered and when it is fully processed. A *pre-order number* is assigned when the traversal first visits a node, before exploring any of its children. A *post-order number* is assigned after all of the node's descendants have been explored and the traversal is about to return to the node's parent. These numbers can be viewed as *timestamps* that describe the progress of the DFS.

Pre-order numbers capture the order in which nodes are discovered, while post-order numbers capture the order in which nodes are completed. This information is useful in many graph and tree algorithms, including dependency analysis, topological sorting (which we will talk about in the future), and determining ancestor-descendant relationships.

Exercise 3 DFS Pseudocode

Working Space

Alter the *recursive* DFS pseudocode to include an assignment of preorder and post order numbers for every node.

Answer on Page 58

Exercise 4 Pre- and Post-order numbers

Working Space

Write the pre and post order numbers for the person graph we have been working with, Figure 6.1

Answer on Page 58

Chapter 7

Finding Shortest Paths

7.1 Introduction — BFS revisited

We have seen BFS as a way to search graphs (or trees). It is a pretty simple process:

- select a start node
- move one distance from the root, and visit all nodes (v) on that level
- when visiting, queue all child nodes (v') of that node.
- keep going until either all nodes are marked *visited* or a provided *target node* is reached.

There is an issue here: what if the graph is weighted, and the paths are not equally weighted? For example, see this graph:

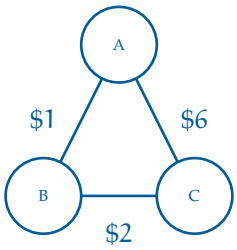


Figure 7.1: A graph of 3 nodes that would fail BFS.

BFS, here, would prefer $A \to C$, since it has an edge count of 1, whereas $A \to B \to C$ has an edge count of 2. However, the true shortest path, by weight, is $A \to B \to C$, with a weight of 3.

So BFS has a major limitation: it cannot handle weighted graphs. This is why we started it out on trees; typically trees do not have a set edge weight, so we were not limited by the order in which it expands.

To handle weighted graphs, we use a new algorithm that expands off of BFS, called *Dijkstra's Algorithm*. Dijkstra's Algorithm avoids the limitation of weights that BFS had by enqueueing the nodes in order of current shortest known distance.

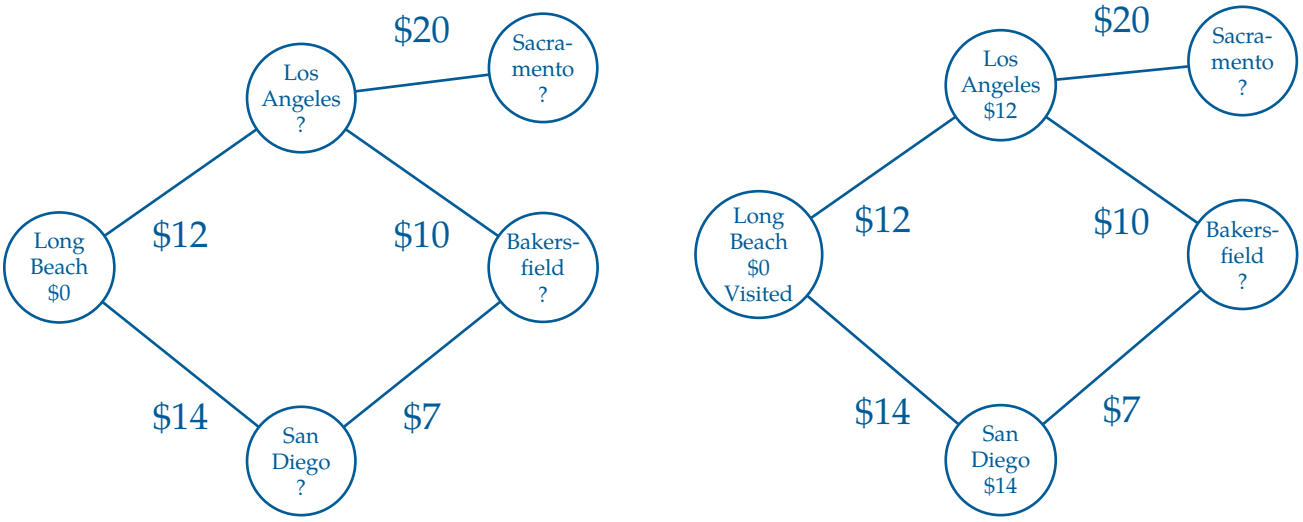
7.2 Dijkstra's Algorithm

Edsger W. Dijkstra (pronounced *DYKE-strah*) was a great Dutch computer scientist. He came up with an algorithm for finding the cheapest path through a graph with weighted edges. Today it is known as *Dijkstra's Algorithm*. It is used in a wide variety of common problems, and is also both simple and elegant.

7.2.1 Algorithm Description

You are going to mark each node with how much it would cost to get there from some chosen origin node. For example, if you are shipping a container from Long Beach, you will mark each city with the cost of getting the container to that city.

You start by marking the price for Long Beach to zero (the container is already there). You then mark each adjacent city with the cost on the edge (figure 7.2a). Next, you declare Long Beach to be "visited" (figure 7.2b).



(a) The initialized state of the graph, selecting Long Beach as our origin node.

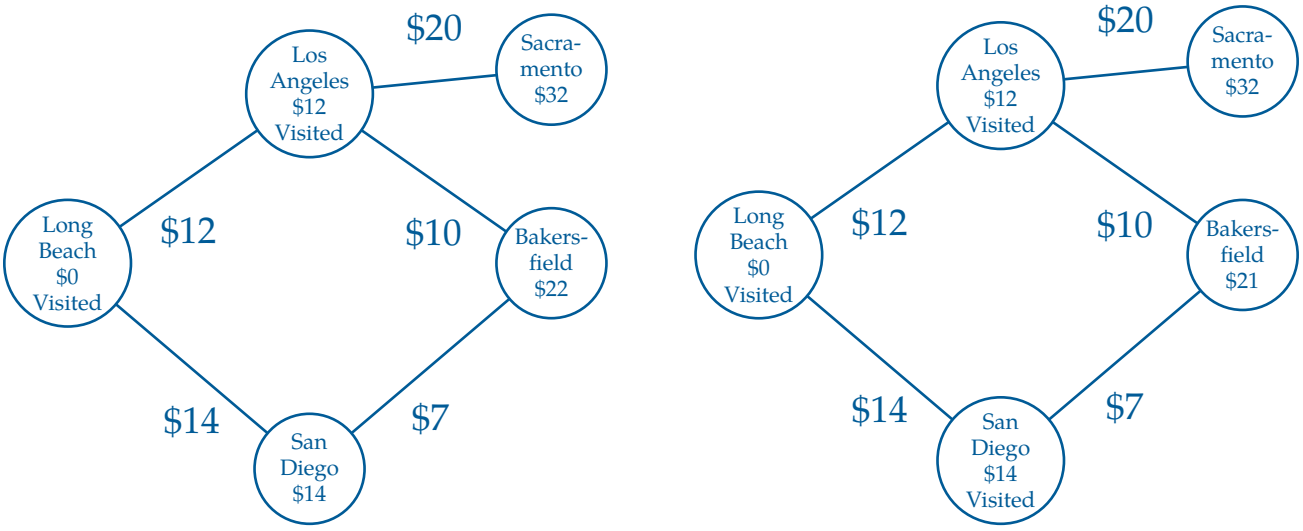
(b) Marking Long Beach as visited, we change the cost values of Los Angeles and San Diego to their respective edges.

Figure 7.2: A map of weighted cities.

After that, you find the cheapest of the unvisited nodes. In this case, Los Angeles is cheaper than San Diego, so that is the node you will visit next.

You mark all of the unvisited nodes adjacent to Los Angeles, with the price to ship it to Los Angeles plus the cost of shipping the container from Los Angeles to that city (figure

7.3a). Note that Bakersfield is marked with $22.



(a) After visiting Los Angeles.

(b) After visiting San Diego.

Figure 7.3: Visiting the next two minimum cost cities.

Now, the cheapest unvisited node is San Diego, so you mark its neighbors with the cost to ship to San Diego, plus the price to ship from San Diego to the neighbor (figure 7.3b).

Notice that Bakersfield is already labeled with $22 from a route through Los Angeles. But the price would be $21 if you shipped it to Bakersfield via San Diego. Because the new route is cheaper, you change the price to the lower value.

(What does it mean that a node is "visited"? If a node is marked visited, it is marked with a price that won't get any smaller.)

You continue visiting the cheapest unvisited node until all the nodes have been visited. Then you know every node has been marked with its lowest price.

In a big graph, each node may be marked several times in this process — each time with a lower price from a cheaper router.

Of course, once you have the price, you will ask, "What is the route that gets me that price?" So, we will also mark each node with the neighbor from which it would receive the shipment — the previous node. This is easy to do as we execute the algorithm.

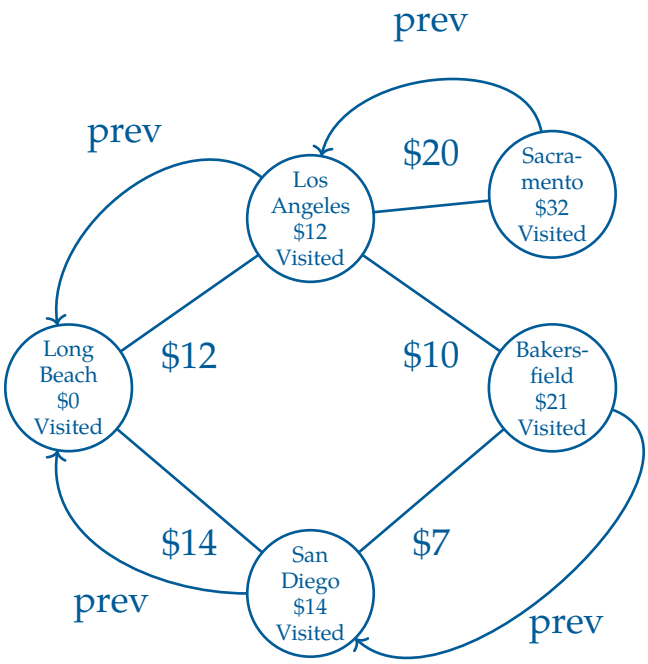


Figure 7.4: Marking each node with a previous pointer.

Now, to figure out the cheapest route from Long Beach to Bakersfield, we start at the destination and follow the `prev` pointer back through San Diego, then to Long Beach.

Exercise 5 Expanding on the Delivery Graph

Notice that the path between Los Angeles to Bakersfield is not used once we add in our previous labels. Why might this be?

Working Space

Answer on Page 58

7.2.2 Implementation

We do not actually want to sully our graph objects with the three additional pieces of information we need:

- The current minimal cost from the origin node. This is usually called the `dist`, from "distance".
- The neighbor who gives us the current minimal cost. This is usually called `prev`, from "previous".
- Whether the city is visited or now.

So, we will keep them in collections external to the graph.

For example, to keep track of the `dist`, we will have a `dist` dictionary. Each node will be a key, the current minimal cost will be the value. If the node hasn't received even a first cost, we will put in infinity as the cost.

(After the algorithm is run, if the cost of a node is still infinity, that means that it cannot be reached from the origin node.)

We will also have a `prev` dictionary. The final node will be the key, and its previous neighbor will be the value.

Finally, the graph has a list of all the nodes, so we can just keep a set of the unvisited nodes.

Add a method to the `Graph` class that implements Dijkstra's algorithm:

```
    def cost_from_node(self, origin_node):
        # Cost of cheapest path from origin node discovered so far
        # Initially the origin is zero and all the other are infinity
        dist = {k: math.inf for k in self.nodes}
        dist[origin_node] = 0.0

        # The previous city on that cheapest path
        prev = {}

        # All the nodes start as unvisited
        unvisited = set(self.nodes)

        # While there are still unvisited nodes
        while unvisited:

            # Find unvisited node with lowest cost
            min_cost = math.inf
```

```
18              for u in unvisited:
19                  if dist[u] < min_cost:
20                      current_node = u
21                      min_cost = dist[u]
22
23              # If none are less than inf, we are done
24              # This happens in graphs that are not connected
25              if min_cost == math.inf:
26                  return (dist, prev)
27
28              # Remove the lowest cost node from the unvisited list
29              unvisited.remove(current_node)
30
31              # Update all the unvisited neighbors
32              for edge in current_node.edges:
33
34                  # What node is at the other end of this edge?
35                  v = edge.other_end(current_node)
36
37                  # Visited nodes are already minimized, skip them
38                  if v not in unvisited:
39                      continue
40
41                  # Is this a shorter route?
42                  alt = dist[current_node] + edge.cost
43                  if alt < dist[v]:
44
45                      # Update the distance and prev dicts
46                      dist[v] = alt
47                      prev[v] = current_node
48
49          return (dist, prev)
```

Each time we visit a node, we have a decision to make:

$$\text{if } \text{dist}[u] + w(u, v) < \text{dist}[v] \text{ then update } \text{dist}[v]$$

This action, the core foundation of Dijkstra's, is done in lines 16-21. In Dijkstra's, the node with the minimum tentative distance can also be finalized, provided all edge weights are non-negative. Nodes left at infinity are not reachable from the chosen root node.

Append some code to your `cities.py` that tests this method:

```
(cost_from_long_beach, prev) = network.cost_from_node(long_beach)
print(f"\nMinimum costs from Long Beach = {cost_from_long_beach}")
print(f"\nLast city before = {prev}")

nyc_cost = cost_from_long_beach[nyc]

if nyc_cost < math.inf:
```

```
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
else:
    print("You can't get to NYC from Long Beach")
```

When you run it, you should get a list of how much it costs to ship a container to each city from Long Beach:

```
Minimum costs from Long Beach = {(node:Long Beach, edges:1): 0.0,
(node:Los Angeles, edges:3): 12.0, (node:Denver, edges:3): 35.0,
(node:Phoenix, edges:3): 31.0, (node:Louisville, edges:3): 49.0,
(node:Cleveland, edges:4): 44.0, (node:Boston, edges:2): 57.0,
(node:New York City, edges:3): 52.0}
```

You will also get a collection of node pairs. What are these? For each node, you get the node that you would pass through on the cheapest route from Long Beach:

```
Last city before = {(node:Los Angeles, edges:3):(node:Long Beach, edges:1),
(node:Denver, edges:3):(node:Los Angeles, edges:3),
(node:Phoenix, edges:3):(node:Los Angeles, edges:3),
(node:Louisville, edges:3):(node:Denver, edges:3),
(node:Cleveland, edges:4):(node:Denver, edges:3),
(node:New York City, edges:3):(node:Cleveland, edges:4),
(node:Boston, edges:2): (node:Cleveland, edges:4)}
```

Your users will not want to read this; give them the shortest path as a list. Add a function to `graph.py` that turns the `prev` table into a path of nodes that lead from the origin to the destination:

```
def shortest_path(prev, destination):

    # Include the destination in the path
    path = [destination]
    current_node = destination

    # Keep stepping backward in the path
    while current_node in prev:

        # What node should come before the current node?
        previous_node = prev[current_node]

        # Insert it at the start of the list
        path.insert(0, previous_node)
        current_node = previous_node

    return path
```

Test that out:

```
if nyc_cost < math.inf:
    print(f"*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"*** Cheapest path from Long Beach to NYC: {path_to_nyc ***}")
else:
    print("You can't get to NYC from Long Beach")
```

This should look like this:

```
*** Cheapest path from Long Beach to NYC: [(node:Long Beach, edges:1),
(node:Los Angeles, edges:3), (node:Denver, edges:3), (node:Cleveland, edges:4),
(node:New York City, edges:3)] ***
\end{text}

% \subsection{Making it faster}

% On really big networks, doing a full Dijkstra's algorithm would take
% too long; luckily, there are many methods for getting similar results
% quickly.  When you ask for directions from Google Maps, it does not do
% a full Dijkstra's Algorithm for every possible route --- it would just
% take too long.

% However, there is a way to speed up this implementation. Look at this snippet:

% \begin{minted}{python}
%             # Find unvisited node with lowest cost
%             min_cost = math.inf
%             for u in unvisited:
%                 if dist[u] < min_cost:
%                     current_node = u
%                     min_cost = dist[u]
```

7.3 Bellman-Ford Algorithm

The Bellman-Ford Algorithm is another shortest path algorithm like Dijkstra's. However, unlike Dijkstra's Algorithm, Bellman-Ford can handle graphs with negative weight edges.

The algorithm works as follows:

1. Assign a tentative distance value for every node: set it to zero for our initial node and to infinity for all other nodes.
2. For each edge (u, v) with weight w, if the current distance to v is greater than the

distance to u plus w, update the distance to v to be the distance to u plus w.

3. Repeat the previous step $|V| - 1$ times, where $|V|$ is the number of vertices in the graph.
4. After the above steps, if you can still find a shorter path, there exists a negative cycle.

If the graph does not contain a negative cycle reachable from the source, the shortest paths are well-defined, and Bellman-Ford will correctly calculate them. If a negative cycle is reachable, no solution exists, but Bellman-Ford will detect it.

Chapter 8

Other Graph Algorithms and Proofs

This chapter is an overview of other graph algorithms, proofs, and concepts that are important to know. We will first cover adjacency matrices, then Prim's algorithm for minimum spanning trees, isomorphism, and topological sorting for directed acyclic graphs (DAGs). A lot of these are very advanced graph theory topics, so you won't be tested on too heavily of these topics. Many of these appear on AP/IB exams, or in advanced college level CS classes.

8.1 Adjacency Matrices

Recall that we can represent graphs as lists of data called adjacency lists. In code, this could look like the following:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

This represents the following graph

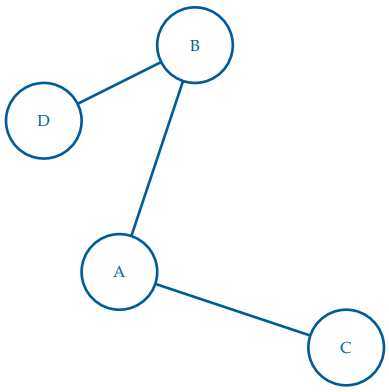


Figure 8.1: A simple (undirected unweighted) graph used to make an adjacency list and adjacency matrix.

We could also represent the same data as a *matrix*, which we have covered in linear algebra. The *adjacency matrix* is 2D array (as compared to an adjacency list, which consists of multiple 1D arrays) where `matrix[i][j]` indicates whether an edge exists from vertex i to vertex j. Commonly, `matrix[i][j]` $= 1$ if and only if the edge from vertex i to vertex j exists, else `matrix[i][j]` $= 0$.[1] The above graph in Figure 8.1 can be represented as:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For *undirected* graphs, the adjacency matrix is symmetric, and thusly $n \times n$, meaning that the transpose of matrix is the same matrix!

$$A = A^T$$

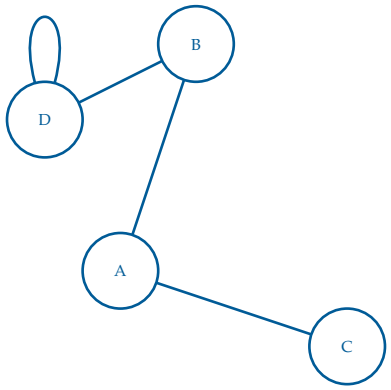


Figure 8.2: A simple graph with an added loop on vertex D.

By convention, a diagonal entry such as `matrix[k][k]`, where $(k \in \mathbb{Z})$, is non-zero only if there is a loop from a vertex to itself. The value of the entry represents the number of such loops. In this case, there are two edges from vertex (D) to itself in Figure 8.2, so the corresponding diagonal entry is (2).

For *directed* graphs, the adjacency matrix on appears a bit different. A *directed adjacency matrix* represents a directed graph, where edges have a direction (arrows). When $A_{ij} = 1$, then an edge exists from vertex i to vertex j.

[1] If the edges are weighted, the matrix[i][j] is equal to the weight, or 0 if the edge does not exist.

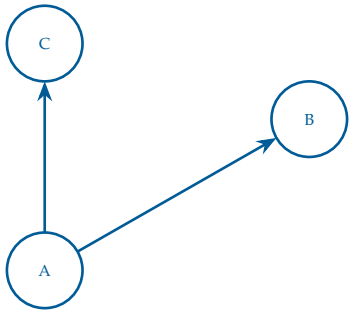


Figure 8.3: A simple directed graph used to make an adjacency matrix.

The resulting adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice how, now, $A \neq A^T$, because the edges are directional.

Exercise 6 Adjacency Matrix Exercise

Working Space

The following is an adjacency matrix for a weighted (undirected) graph.

	A	B	C	D	E	F	G
A	0	4	2	0	0	0	0
B	4	0	1	5	0	0	0
C	2	1	0	8	10	0	0
D	0	5	8	0	2	6	0
E	0	0	10	2	0	3	7
F	0	0	0	6	3	0	4
G	0	0	0	0	7	4	0

Draw the corresponding graph with weighted edges.

Answer on Page 59

8.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees

Prim's Algorithm is minimum spanning tree

FIXME https://github.com/KontinuaFoundation/sequence/issues/315

8.3 A* (A-star) Search Algorithm

The last shortest path we will discuss in this chapter is the *A-star Algorithm*, more commonly represented as *A* Algorithm*. In essence, it is an extension of Dijkstra's Algorithm in combination with a *priority-based heuristic function*. A heuristic is an introduced estimated cost from any node to the goal.

The key behind this implementation is the following function

$$f(n) = g(n) + h(n)$$

where

- $g(n)$: the cost of the path from the start node (usually S) to the current node n
- $h(n)$: the *heuristic* function that estimates the remaining cost from n to the goal (usually E)
- $f(n) = g(n) + h(n)$: the estimated total cost of a path from S to E passing through node n

8.3.1 The Path Cost $g(n)$

$g(n)$ remains to be the same value that we used in Dijkstra's and other shortest path algorithms.

When moving from a node u to an adjacent node v, the path cost is *updated* as

$$g(v) = g(u) + c(u, v)$$

where $c(u, v)$ is the cost of traversing the edge from u to v. As we discussed in the chapter on Dijkstra's, we have a decision to make upon each visit to a node: Each time we visit a node, we have a decision to make:

$$\text{if } dist[u] + w(u, v) < dist[v] \text{ then update } dist[v]$$

8.3.2 The Heuristic Function $h(n)$

The heuristic $h(n)$ function depends on the problem domain. Its purpose is to estimate the remaining cost from node n to the goal. While this is a difficult task, we can use basic algebra to make an informed guess.

For example, in a map-based pathfinding problem, $h(n)$ may be the straight-line distance, or *Euclidean distance* between node n and the goal. Since the actual path cannot be shorter than the Euclidean distance, this provides a reasonable estimate of the remaining cost. If a nodes location is represented on a 2D plane (such as a map or graph) (x_n, y_n) and the goal location as (x_g, y_g), the Euclidean Heuristic

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$$

Alternatively, many 2D games use a similar either *taxicab distance* (typically for 4-way movement),

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

or *Chebyshev distance* (typically for 8-way movement in a game)

$$d = \max(|x_2 - x_1|,\ |y_2 - y_1|)$$

which takes the maximum of the horizontal and vertical movements.

Sometimes, graphs without direct spatial meaning become more difficult to work with because there is no obvious notion of distance between two nodes.

For general graphs, such as social networks, communication networks, or dependency graphs, the heuristic must be derived from problem-specific knowledge. For example, a social network application might use information such as mutual connections, community membership, or other metadata to estimate how *close* a user is to a target user.

In many cases, however, no suitable heuristic is known. In these situations, the heuristic is often defined as

$$h(n) = 0$$

for all nodes n. Substituting this into the A* evaluation function gives

$$f(n) = g(n)$$

which causes A* to reduce to to Dijkstra's algorithm. While this guarantees an optimal path, it loses the efficiency gains that a good heuristic can provide.

There is an important property that the Heuristic Function must satisfy:

$$h(n) \leq \text{true remaining cost}$$

When an admissible heuristic is used (such that $h(n) \leq$ true remaining cost), A* is guaranteed to find an optimal path if one exists.

8.3.3 The Total Estimated Distance $f(n)$ and Priority Queues

The purpose of the function

$$f(n) = g(n) + h(n)$$

is to assign a priority to each node during the search. Recall from our discussion of priority queues that each element is associated with a priority value, and the element with the highest (or lowest) priority is removed first.

In A*, the priority of a node is its estimated total distance $f(n)$. Nodes with smaller values of $f(n)$ are considered more promising because they represent paths that appear likely to reach the goal with a lower total cost.

As with Dijkstra's algorithm, A* maintains a priority queue containing nodes that have been discovered but not yet fully explored. However, instead of prioritizing nodes solely by their known distance from the start node, A* prioritizes nodes according to their estimated total distance

$$\text{priority}(n) = f(n)$$

At each step, the node with the smallest $f(n)$ value is removed from the priority queue and explored. When a neighboring node is discovered, its path cost $g(n)$ is updated, a new heuristic estimate $h(n)$ is calculated, and a new value of $f(n)$ is computed.

There are some good references about this in your digital resources, as well as a fun Minecraft-based explanation here.

8.4 Injection, Surjection, Bijection, and Isomorphism

Let's talk about Graph Isomorphism. To do this, we must first introduce the terms of *injection*, *surjection*, and *bijection*. These are types of functions in set theory as well as mathematics.

Definition 1 (Injection)**.** *A function $f : A \to B$ is injective if different inputs always produce different outputs. Importantly, this means no two integers map to the same output.*

Definition 2 (Surjection)**.** *A function $f : A \to B$ is surjective if every element of B is hit by at*

least one element of A. In essence, all of the elements of B are covered.

Definition 3 (Bijection)**.** *A function* $f : A \to B$ *is bijective if it is both injective and surjective. Ultimately, the function creates no collisions and hits all functions of B.*

Exercise 7 Bijective, Surjective, Injective Exercise

Working Space

Determine whether each function is injective, surjective, bijective, or none of these.

1. $f : \{1, 2, 3\} \to \{a, b, c, d\}$ defined by

   $f(1) = a, \quad f(2) = b, \quad f(3) = c.$

2. $g : \{1, 2, 3, 4\} \to \{a, b, c\}$ defined by

   $g(1) = a, \quad g(2) = b, \quad g(3) = c, \quad g(4) = a.$

3. $h : \{1, 2, 3\} \to \{a, b, c\}$ defined by

   $h(1) = b, \quad h(2) = c, \quad h(3) = a.$

Answer on Page 59

Now, let's talk about *Isomorphism*. A *graph isomorphism* is a bijection between the vertex sets of two graphs that preserves adjacency. Thus, an isomorphism not only matches vertices one-to-one, but also preserves the structure of the graph.

Definition 4 (Graph Isomorphism)**.** *Let* $G = (V_G, E_G)$ *and* $H = (V_H, E_H)$ *be graphs. An isomorphism from* G *to* H *is a bijection*

$$f : V_G \to V_H$$

such that for all vertices $u, v \in V_G$*,*

$$\{u, v\} \in E_G \quad \iff \quad \{f(u), f(v)\} \in E_H.$$

If such a function exists, we say that G *and* H *are isomorphic and write*

$$G \cong H.$$

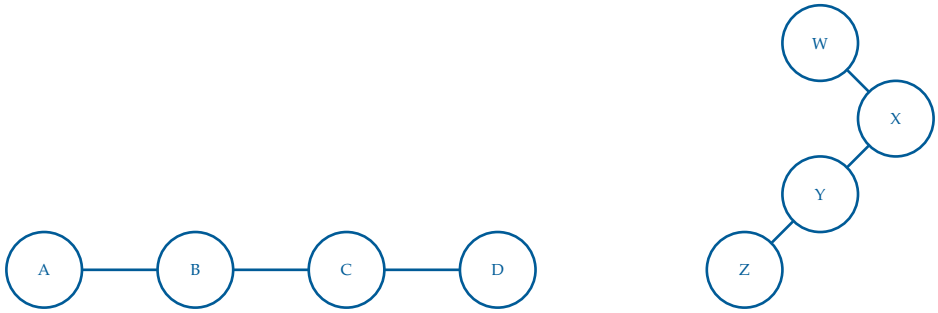


Figure 8.4: Two graphs that are isomorphic. Graph G (left) and Graph H (right).

We are able to say that these graphs (Figure 8.4) are isomorphic as one vertex from G can be paired with one vertex from H, while maintaining the same edge connections.

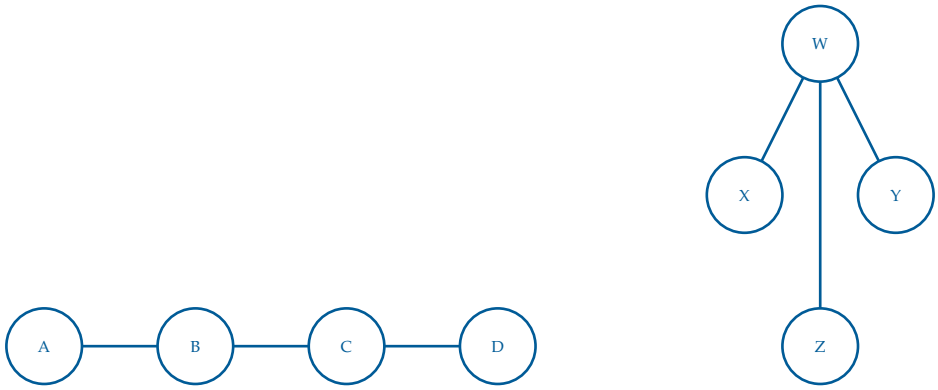


Figure 8.5: Graph X and Y; not isomorphic.

In Figure 8.5, we can see the vertices are not isomorphic without adding or subtracting edges. For example, Graph X does not have a vertex with 3 edges, unlike Graph Y.

If any of these differ, the graphs are not isomorphic:

- Number of vertices
- Number of edges
- Degree sequence (if degree sequences are intact, it does not guarantee isomorphism)
- Number of connected components
- Number of cycles of a given length

- Chromatic number

Example.

A common example that you will not be tested on but is important to know is the following. We may need to introduce a definition first.

Definition 5 (Clique)**.** *A clique is a set of vertices in a graph such that every pair of distinct vertices is connected by an edge.*

Definition 6 (Clique Subgraph)**.** *A* k*-clique is exactly a subgraph isomorphic to the complete graph* K_k*. A 3-clique would be a clique of size 3, called* K_3*.*

Now, given two graphs G and H, how can we determine G contain a subgraph isomorphic to H, a graph of smaller to equal size to G?

This may be easy if G and H are pretty small, but the math becomes infinitely harder as the number of vertices and edges go up.

We can simplify this by transforming the inputs of a clique problem to inputs of the subgraph isomorphism problem.

The clique problem gives us a graph M and a threshold k. The output of this is whether M contains a clique of size at least k.

To transform this into an instance of the subgraph isomorphism problem, construct a smaller subgraph $H = K_k$, the complete graph on k vertices, and let $G = M$, the larger graph.

We now ask the subgraph isomorphism question:

> Does G contain a subgraph isomorphic to H?

Since H is the complete graph K_k, a subgraph of G isomorphic to H must contain k vertices where every pair of vertices is connected by an edge. By definition, this is exactly a k-clique.

Therefore,

$$M \text{ contains a } k\text{-clique} \iff G \text{ contains a subgraph isomorphic to } K_k.$$

This shows that every instance of the Clique problem can be transformed into an instance of the Subgraph Isomorphism problem. We call this an *algorithmic reduction*.[2]

[2] Algorithmic Reductions are the basis of a set of problems called P, NP, and NP-Complete. See this video

8.5 Travelling Salesperson Problem and Nearest Neighbour

FIXME https://github.com/KontinuaFoundation/sequence/issues/312

8.6 Topological Sort and DAGs

Another common use of graph algorithms is sorting them in a meaningful order. Specifically, we can sort directed graphs that tells us exactly what order vertices need to be "visited" or operated on. We call this a *Topological Sorting* of the graph. A *topological order* is a linear ordering of the vertices in a graph such that for every directed edge $u \to v$ from vertex u to vertex v, vertex u comes before vertex v in the ordering. The most common usage for this is scheduling jobs, queueing tasks with dependencies, or creating tentative project schedules.

To do this, we are going to use Python's `Graphlib`, and its `TopologicalSorter` module.

In the `graphlib` module, graphs are defined in the following format:

```
{
  node: {predecessors}
}
```

A basic example of this is the following

```
{
  "C": {"A", "B"}
}
```

Which says that A and B need to come before C, such that one possible ordering is

$$A \to B \to C$$

Now suppose we alter the graph format as follows:

```
{
  "C": {"A", "B"},
  "A": {"Z"},
  "B": {"Z"},
  "Z": {"C"}
}
```

for more.

Here,

- C depends on A and B
- A and B depend on Z
- Z depends on C

There is no way to determine a *starting point*. Now we have an ordering that contains a cycle. This implies that C must come before itself, which is impossible in a linear ordering. Therefore no topological ordering can exist.

Definition 7 (Cycle)**.** *A cycle is a valid walk* $(v_1, v_2, \ldots, v_n)$ *where the only repeated vertex is* $v_n = v_1$*, meaning the last vertex equals the first vertex.*

So, we cannot do a topological sort on a graph that contains a cycle. This provides us with a theorem:

Topological Sorting and DAGs

A directed graph admits a topological ordering if and only if it is a *Directed Acyclic Graph* (DAG). That is, the directed graph may not contain any cycles (acyclic).

In fact, the python graphlib library will not produce a valid Topological Sort if a cycle exists. Running the following code

```
from graphlib import TopologicalSorter
graph = {
  "C": {"A", "B"},
  "A": {"Z"},
  "B": {"Z"},
  "Z": {"C"}
}
list(TopologicalSorter(graph).static_order())
```

will produce the following error

```
graphlib.CycleError: ('nodes are in a cycle', ['C', 'Z', 'A', 'C'])
```

The exception reports one cycle that prevents the ordering from being constructed. In this case, $C \to Z \to A \to C$ forms a directed cycle.

We can also introduce tasks that can be done in parallel. Create the following file `parallel_dag.py`

```
from graphlib import TopologicalSorter
tasks = {
    "Compile": set(),
    "Build Frontend": {"Compile"},
    "Build Backend": {"Compile"},
    "Test": {"Build Frontend", "Build Backend"},
    "Deploy": {"Test"},
}from graphlib import TopologicalSorter
tasks = {
    "Compile": set(),
    "Build Frontend": {"Compile"},
    "Build Backend": {"Compile"},
    "Test": {"Build Frontend", "Build Backend"},
    "Deploy": {"Test"},
}
from graphlib import TopologicalSorter

ts = TopologicalSorter(tasks)
ts.prepare()

while ts.is_active():
    ready = ts.get_ready()
    print(ready)

    for task in ready:
        ts.done(task)

```

The 5 nodes here are tasks for a basic software project. The ordering is as follows:

$$\text{Compile} \rightarrow \text{Build Frontend and Build Backend} \rightarrow \text{Test} \rightarrow \text{Deploy}$$

However notice that *Build Frontend* and *Build Backend* are not dependent on each other, so they could be done in *parallel*. The *Test* task cannot begin until both builds have finished. Running this code outputs the following

```
('Compile',)
('Build Frontend', 'Build Backend')
('Test',)
('Deploy',)
```

showing us that our sort outputs which ones can happen at the same time. In software development, many tasks have dependencies but can still be executed in parallel when those dependencies are satisfied. Topological sorting helps determine both the required execution order and which tasks may be performed concurrently (this is similar to running circuits in parallel!).

8.7 Proofs

This section will be a set of proofs or algorithm design problems for graph-related concepts that are important to know. At the basis level, these are just practice problems for graph understand, but can be applied in many real world scenarios.

8.7.1 Example 1: Bipartite

Bipartite Graph

A *Bipartite Graph* is a graph $G = (V, E)$ such that V can be partitioned into two sets $(V = V_1 \cup V_2)$ and $(V_1 \cap V_2 = empty)$ such that there are no edges between vertices in the same set. FIXME Diagram

Theorem 1. *An undirected graph* G *is bipartite if and only if it contains no cycles of odd length.*

Exercise 8 Algorithm for Determining Bipartite

Working Space

Design pseudocode for an algorithm that determines whether or not an undirected graph is bipartite. *Hint: use the theorem.*

Hint 2: This function may be useful

`FindOdd`(G, e)

Require: G, a graph on which DFS has been run; $e = (x, y)$, a back edge
Ensure: **true** if e creates a cycle of odd length, **false** otherwise

```
1: if pre(x)−post(y) ≡ 0 (mod 2) then
2:     return true
3: else
4:     return false
5: end if
```

Answer on Page 60

While you don't have to go through it on your own, here is the outline of how we can do a proof by contradiction for a graph, specifically proving theorem 1.

Proof. First, suppose G is bipartite. Then its vertices can be partitioned into two sets V_1 and V_2, with every edge going between V_1 and V_2. Any cycle in G must alternate between vertices in V_1 and vertices in V_2. Therefore, after an odd number of edges, the cycle would end in the opposite set from where it started, so it cannot return to its starting vertex. Hence every cycle has even length.

Conversely, suppose G contains no cycles of odd length. Choose any connected component of G, and pick a vertex s. Put each vertex at even distance from s in V_1, and each vertex at odd distance from s in V_2. We claim this gives a valid bipartition.

Assume for contradiction that there is an edge (u, v) where u and v are in the same set. Then u and v have distances from s with the same parity. Taking shortest paths from s to u and from s to v, together with the edge (u, v), creates a cycle of odd length. This contradicts the assumption that G has no odd cycles.

Thus no edge connects two vertices in the same set, so each connected component is bipartite. Applying this construction to every connected component of G, the whole graph G is bipartite.

Therefore, G is bipartite if and only if it contains no cycles of odd length. □

8.7.2 Example 2 and 3. Cycle Detection and Leaves

Exercise 9 Cycle Detection

Design pseudocode for an algorithm which takes as an input an undirected graph, G, and an edge, $e = (u, v)$, and determines whether G has a cycle containing e.

Working Space

Answer on Page 60

Exercise 10 Leaf Removal

Proof the following statement: In any connected undirected graph, there is a vertex whose removal leaves the graph connected.

Working Space

Answer on Page 60

Sometimes, we may need to create new graphs as counterexamples to disprove a statement. Consider the following:

Theorem 2. *An graph* G *is strongly connected if every vertex is reachable from every other vertex.*

Another way to say this is that a select vertex of the graph has a path to every other vertex on the graph. Often, this comes up in finding *subgraphs* of graphs that are strongly connected, called *strongly connected components*.

Exercise 11 Strongly Connected Graphs

Create an example of a strongly connected directed graph $G = (V, E)$ such that, for every $v \in V$, removing v from G leaves a directed graph that is not strongly connected.

Hint: the simplest answer is usually the best. Start with one vertex.

Working Space

Answer on Page 61

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 21)

The path to `data.csv` is

```
./Documents/Projects/project2/data.csv
```

The depth is the number of edges from the root to the node.

In this case, it is the number of subdirectories needed to reach `data.csv`:

(1) `./` → `Documents`

(2) `Documents` → `Projects`

(3) `Projects` → `project2`

(4) `project2` → `data.csv`

So `data.csv` is at a depth of **4**.

Answer to Exercise 2 (on page 29)

Algorithm: DFS (Pre-order) run recursively on a tree

```
procedure DFS(root)
    if root = null then
        return
    end if
    visit(root)                ▷ V
    DFS(root.left)             ▷ L
    DFS(root.right)            ▷ R
end procedure
```

This algorithm defines a depth-first search on a binary tree. Because the node is visited before its subtrees, it corresponds specifically to a pre-order traversal.

Answer to Exercise 3 (on page 30)

```
procedure DFS(root)
    if root = null then
        return
    end if
    pre(root) = counter
    counter = counter + 1
    DFS(root.left)
    DFS(root.right)
    post(root) = counter
    counter = counter + 1
end procedure
```

Answer to Exercise 4 (on page 30)

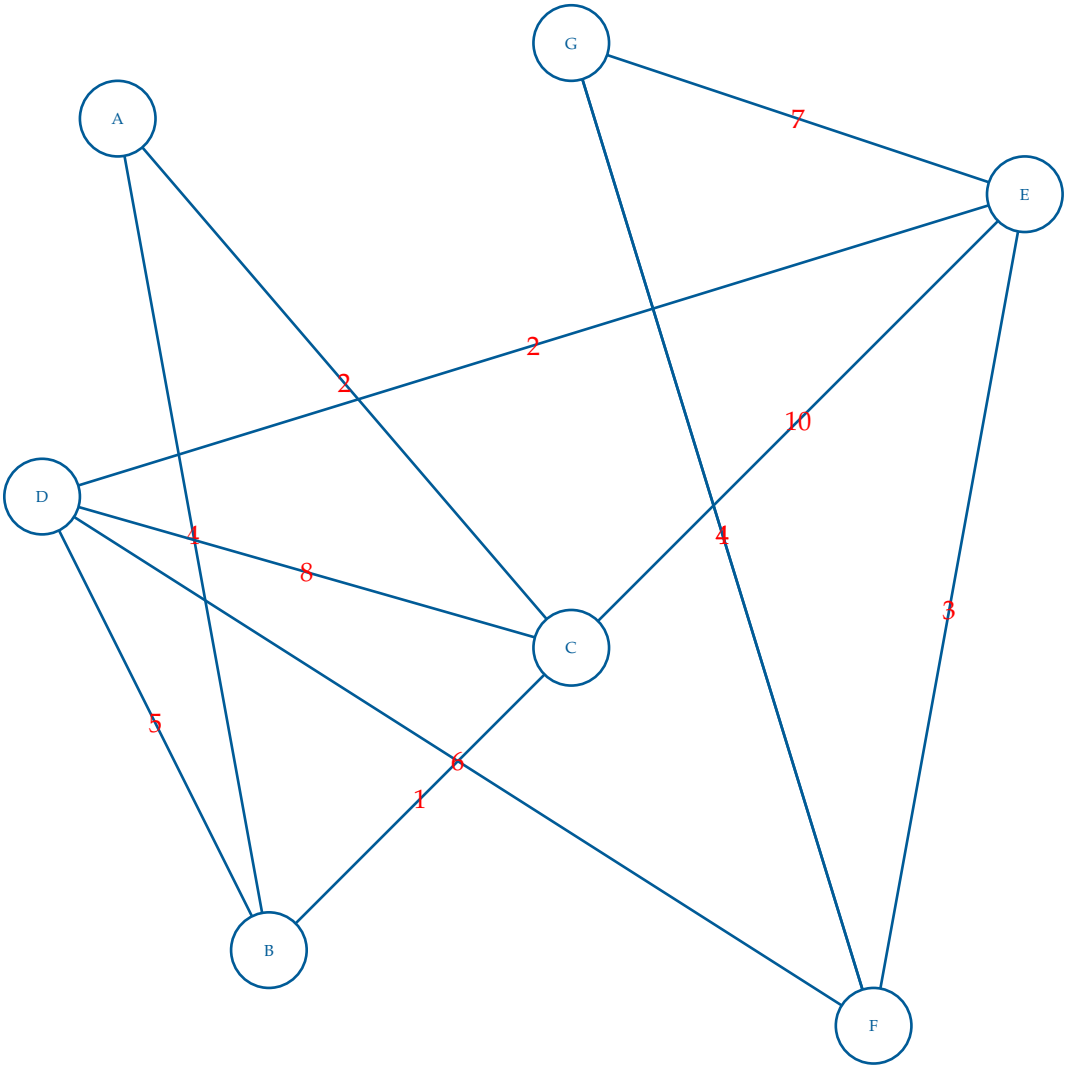
Node	Pre-order Number	Post-order Number
Alan	1	14
Brad	2	7
Dean	3	4
Emily	5	6
Connie	8	13
Faith	9	10
Gordon	11	12

Answer to Exercise 5 (on page 34)

Los Angeles $\rightarrow$ Bakersfield is not chosen because it does not give the best known cost to Bakersfield. The `prev` pointer for Bakersfield goes to San Diego instead, since \$21 $<$ \$22.

The tree that is generated is called a *predecessor tree* or *explored tree* of the resulting edges formed by only the previous edges. We can use this to track shortest paths from a given origin node.

Answer to Exercise 6 (on page 43)



Answer to Exercise 7 (on page 47)

1. Injective, but not surjective. No two inputs map to the same output, but d is never reached.
2. Surjective, but not injective. Every element of the codomain is reached, but both 1 and 4 map to a.
3. Bijective. Every element of the codomain is reached exactly once.

Answer to Exercise 8 (on page 53)

Assuming `FindOdd` exists, we can make the following pseudocode.

`IsBipartite`(G)
Require: G, a graph on which DFS has been run
Ensure: **true** if G is bipartite, **false** otherwise
1: **for** all back edges e **do**
2: **if** FindOdd$(G, e) =$ **true then**
3: **return false**
4: **end if**
5: **end for**
6: **return true**

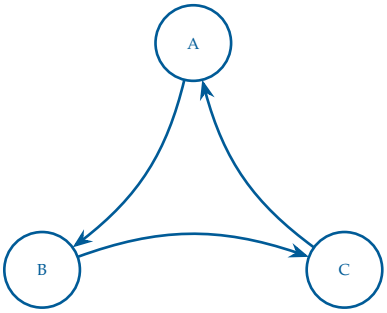
Answer to Exercise 9 (on page 54)

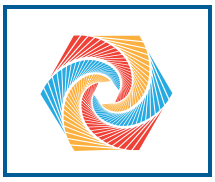
Remove the edge $e = (u, v)$ from G.
Run BFS or DFS starting from u.
if v is reached **then**
Return True.
else
Return False.
end if

Answer to Exercise 10 (on page 55)

Proof. Run DFS on given graph G. Find a leaf, l in the output DFS tree (sometimes called the explore tree). Remove l from the graph. Run DFS on the graph $(G' = G/\{l\})$. If the DFS tree is connected, then we are done. If not, then there is a vertex v in the DFS tree that is not connected to the rest of the graph. This means that v was only connected to l in the original graph. But this contradicts the fact that l was a leaf in the DFS tree, since v would have been explored before l. Since a leaf must exist, removing it from the explore tree does not create a new tree. Therefore, there must be a vertex whose removal leaves the graph connected. □

Answer to Exercise 11 (on page 55)





Index

A* Algorithm, 44
A-star Algorithm, 44
adjacency matrix, 42
algorithmic reduction, 49
Ancestor, 18

Bellman-Ford algorithm, 38
BFS, 23
bijection, 46
Binary Search, 12
binary tree, 18
Bipartite Graph, 53
breadth-first search, 23

Chebyshev distance, 45
collisions in hash tables, 15
cycle, 51
Cycle Error, 51

depth, 18
depth-first search, 23
Descendant, 18
DFS, 23
Dijkstra's Algorithm, 31, 32
Dijkstra, Edsger, 32
Directed Acyclic Graph, 51
directed adjacency matrix, 42
directory, 20

edges, 17
Euclidean distance, 45
explored tree, 58

FIFO, 7, 24
folder, 20
forest, 17

graph isomorphism, 47

hash function, 15
hash table, 15
height, 18
heuristic, 44

in-order, 23
injection, 46
Isomorphism, 47
iterative, 28

leaves, 18
left child, 18
level, 18
level-order, 23
LIFO, 3

matrix, 42

Path, 18
post-order, 23
post-order number, 29
pre-order, 23
pre-order number, 29
predecessor tree, 58
priority queue, 9

queue, 24

recursive, 17, 28
recursive traversal, 28
right child, 18
root, 17

siblings, 17
stack, 3
strongly connected, 55
strongly connected components, 55
subtree, 17
surjection, 46

taxicab distance, 45
top, 3
topological order, 50
Topological Sorting, 50
traversals, 23
tree, 17